THESIS

PAUDEL DIPAK

DEBRECEN
2026

UNIVERSITY OF DEBRECEN
FACULTY OF INFORMATICS

THESIS

Developing High-Level AI Architectures and Systems for Industrial
Defect Detection using Transformers, VAEs, and Diffusion Models

Supervisor:
Robert Lakatos
Assistant Professor

Candidate:
PAUDEL DIPAK
Data Science, MSc

DEBRECEN
2026

Declaration of Authorship

I, Paudel Dipak (name), C20ON7 (Neptune code), hereby declare that this thesis entitled *"Developing High-Level AI Architectures and Systems for Industrial Defect Detection using Transformers, VAEs, and Diffusion Models"* is the result of my own independent research and has been written by me.

I confirm that:

- This research was carried out during my enrolment as a Data Science, MSc. student at the Faculty of Informatics, University of Debrecen.
- Any previously submitted academic material included in this thesis has been clearly identified and properly acknowledged.
- All sources of information, including books, articles, and electronic resources, have been cited in accordance with the IEEE referencing standard.
- All direct quotations from other authors have been clearly indicated, and all other content represents my own original work.
- Every individual and resource that contributed to this work has been appropriately acknowledged.

I take full responsibility for the content of this thesis.

Debrecen, Date: 08.04.2026

Signed: _____
PAUDEL DIPAK
C20ON7

Declaration on the Use of Artificial Intelligence Tools

In preparing this thesis, I made use of artificial intelligence-based tools in accordance with §22 of the Faculty of Informatics Annex of the Academic and Examination Rules and Regulations of the University of Debrecen.

The following AI tools were used:

Tool name: ChatGPT (developed by OpenAI), **Version:** GPT-4o (free tier, access 2025–26)
Purpose of use: Assistance with explaining programming issues (e.g., resolving NaN errors), language refinement, and improving clarity of written content

Tool name: Gemini (developed by Google), **Version:** Gemini 1.5 (free tier, access 2025–26)
Purpose of use: General conceptual clarification and minor writing assistance

Tool name: Claude (developed by Anthropic), **Version:** Claude Sonnet 4.5 (free tier access 2025–26)
Purpose of use: Structural suggestions, paraphrasing, and academic writing improvements

Tool name: Grammarly, **Version:** Free version
Purpose of use: Grammar correction, spelling, and sentence polishing

I confirm the following:

- All technical content, experimental results, trained models, source code, data analysis, and intellectual contributions are entirely my own original work.
- AI tools were used solely for assistance in debugging, language improvement, and formatting; they did not generate any original research contributions, results, or conclusions.
- All outputs from AI tools were carefully reviewed, verified, and modified where necessary.
- I take full personal responsibility for the accuracy, authenticity, and completeness of all content in this thesis.
- The use of these tools does not reduce my meaningful intellectual contribution to the work presented.

Debrecen, Date: 08.04.2026

Signed: Dipak
PAUDEL DIPAK

Acknowledgements

First and foremost, I would like to express my sincere gratitude to Dr. Robert Lakatos, Assistant Professor at the Faculty of Informatics, University of Debrecen, for giving me the opportunity to undertake this research under his supervision. His consistent guidance, constructive feedback, and willingness to address the practical challenges I encountered throughout this project were invaluable. This work would not have reached its current form without his support.

I would also like to thank the Faculty of Informatics, University of Debrecen, for providing the computational infrastructure and academic environment that made this research possible. The resources made available during this programme greatly supported the training and evaluation carried out in this thesis.

I am deeply grateful to the open-source communities behind PyTorch, the timm library, and Gradio, whose tools formed the technical backbone of this project. I would also like to acknowledge Northeastern University, China, for making the NEU Steel Surface Defect Database publicly available, which served as the experimental foundation of this work.

I would like to thank my fellow students in the MSc Data Science programme for their encouragement, insightful discussions, and timely support throughout this journey.

Last but not least, I would like to express my heartfelt gratitude to my family, whose continuous support, patience, and encouragement made it possible for me to complete this thesis.

Abstract

Automated detection of surface defects in steel manufacturing is a challenging task due to the rarity and variability of defects. This thesis proposes a hybrid artificial intelligence framework that integrates a Vision Transformer (ViT-B/16) for classification, a β-Variational Autoencoder (β-VAE) for anomaly localisation, and a Denoising Diffusion Probabilistic Model (DDPM) for synthetic data generation. A fusion classifier combines the outputs of these models to produce accurate and interpretable predictions. Experiments on the NEU Steel Surface Defect Dataset demonstrate that the system achieves 100% classification accuracy with a weighted F1-score of 1.0000. The anomaly detection module generates meaningful heatmaps that highlight defect regions and reflect their severity. The system is deployed as a real-time web application, confirming its practical applicability in industrial environments.

Table of Contents

Declaration of Authorship i
Declaration on the Use of Artificial Intelligence Tools ii
Acknowledgements iii
Abstract iv
List of Figures viii
List of Tables ix
Chapter 1: Introduction 1
1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Research Objectives 3
1.4 Principal Contributions 3
1.5 Thesis Structure 4
Chapter 2: Literature Review 5
2.1 Classical Approaches to Surface Defect Detection 5
2.2 Deep Learning for Visual Inspection 5
2.3 Vision Transformers in Industrial Inspection 6
2.4 Variational Autoencoders for Anomaly Detection 7
2.5 Denoising Diffusion Probabilistic Models 8
2.6 Hybrid Discriminative–Generative Frameworks 9
2.7 The NEU Steel Surface Defect Dataset 10
Chapter 3: Dataset and Preprocessing 11
3.1 Dataset Selection 11
3.2 Dataset Structure and Class Composition 11
3.3 Data Partitioning Strategy 12
3.4 Preprocessing Pipelines 12
3.4.1 ViT Training Transforms 13
3.4.2 ViT Validation and Test Transforms 13
3.4.3 VAE Transforms 13
3.5 Dataset Compatibility Layer 13
Chapter 4: System Architecture 15
4.1 Overview of the Tri-Component Pipeline 15
4.2 Vision Transformer Component 15
4.3 Progressive Unfreezing Strategy 16
4.4 Beta-Variational Autoencoder 17

4.5 DDPM U-Net Architecture 17
4.6 Fusion Classifier 18
4.7 Anomaly Heatmap Generation 18
4.8 Architectural Rationale: Why Three Components? 19
Chapter 5: Implementation and Methodology 21
5.1 Development Environment 21
5.2 ViT Training Protocol 21
5.3 β-VAE Training Protocol 22
5.4 DDPM Training Protocol 22
5.5 Fusion Classifier Training 22
5.6 NaN Instability Resolution 23
Chapter 6: Experimental Results and Evaluation 25
6.1 ViT Training Dynamics 25
6.2 β-VAE Training Dynamics 26
6.3 DDPM Training Dynamics 26
6.4 Fusion Classifier Results 27
6.5 Per-Class Performance Analysis 28
6.6 Live Gradio Inference Evaluation 29
6.7 Generation Mode Validation 29
Chapter 7: Deployment with Gradio 30
7.1 Application Architecture 30
7.2 Detection Mode 30
7.3 Generation Mode 31
7.4 Hugging Face Spaces Deployment 31
Chapter 8: Discussion 32
8.1 Interpreting the Results 32
8.2 Component-Level Contributions 32
8.3 Limitations 33
8.4 Comparison with Published Benchmarks 34
Chapter 9: Conclusion and Future Work 35
9.1 Summary of Contributions 35
9.2 Key Findings 35
9.3 Future Research Directions 36
9.4 Concluding Remarks 37
References 39

Appendix A: Model Architecture Specifications 41
A.1 ViT-B/16 Architecture 41
A.2 β-VAE Architecture 41
A.3 DDPM U-Net Architecture 42
Appendix B: Complete Training Configuration 43
B.1 Hardware and Software Environment 43
B.2 Dataset Split Configuration 43
B.3 Checkpoint Inventory 44
Appendix C: Gradio Application Screenshots 45
Appendix D: Key Implementation Code 48
D.1 β-VAE Reparameterisation and ELBO Loss 48
D.2 DDPM Forward Diffusion (q_sample) 48
D.3 Anomaly Score Computation 48
D.4 Progressive Unfreezing — Complete Optimiser Rebuild 48
D.5 NEU-DET Artefact Masking at Inference 48

List of Figures

Fig. 1 Tri-Modal Defect Detection Pipeline 15
Fig. 2 ViT-B/16: Patch Tokenisation to Classification Head 7
Fig. 3 β-VAE Encoder–Decoder and Anomaly Detection Principle 8
Fig. 4 DDPM: Forward Noising and Reverse Denoising Process 9
Fig. 5 Fusion Classifier: Three-Stream Architecture 18
Fig. 6 Three Normalisation Pipelines (Must Not Be Mixed) 12
Fig. 7 Progressive Unfreezing Strategy for ViT Fine-Tuning 16
Fig. 8 Two Critical NaN Instabilities: Diagnosis and Resolution 23
Fig. 9 Eight-Stage Anomaly Heatmap Generation Pipeline 19
Fig. 10 ViT-B/16 Training Loss and Accuracy Curves 25
Fig. 11 β-VAE ELBO Convergence Curve 26
Fig. 12 DDPM Training Loss Plateau 27
Fig. 13 Confusion Matrix — Fusion Classifier 28
Fig. C.1 Gradio — Crazing Detection Mode 45
Fig. C.2 Gradio — Crazing Generation Mode 46
Fig. C.3 Gradio — Crazing Detection Mode 46
Fig. C.4 Gradio — Crazing Generation Mode 47

List of Tables

Table 1 NEU Steel Dataset Class Composition 11
Table 2 Software Libraries and Framework Versions 21
Table 3 ViT Training Hyperparameters — Phase 1 and Phase 2 22
Table 4 ViT Epoch-by-Epoch Training Milestones 25
Table 5 Fusion Classifier Test Results Summary 27
Table 6 Per-Class Classification Report on NEU Steel Test Set 28
Table 7 Live Gradio Inference Results — Both Evaluation Sessions 29
Table A.1 ViT-B/16 Architecture Specifications 41
Table A.2 β-VAE Architecture Specifications 41
Table A.3 DDPM U-Net Architecture Specifications 42
Table B.1 Hardware and Software Environment 43
Table B.2 Dataset Split Configuration 43
Table B.3 Checkpoint Inventory 44

Chapter 1: Introduction

1.1 Background and Motivation

Modern steel production lines operate at speeds that make unaided human surface inspection structurally unreliable. Strip metal passes through a rolling mill at rates where critical defects — fine crazing networks, sub-millimetre inclusions, shallow pitted cavities — appear and disappear from any inspection window within fractions of a second. Fatigue accumulates across shifts, lighting conditions vary between stations, and two trained inspectors examining the same surface will sometimes reach different conclusions about whether a particular marking constitutes a defect.

The first generation of automated inspection systems, developed from the early 1980s, encoded expert judgment as hand-crafted mathematical rules: Gabor filter banks tuned to specific spatial frequencies, grey-level co-occurrence matrices [1] capturing second-order intensity statistics, and morphological operators shaped to match expected defect geometries. These systems worked within a narrow operational envelope. When the steel grade changed, when rolling temperature altered the background texture, or when a new surface specification was required, the inspection rules needed rewriting — an engineering effort measured in weeks, not hours.

Machine learning shifted the bottleneck: Support Vector Machines combined with Local Binary Patterns, HOG features, or SIFT keypoints [2] produced more adaptable classifiers. But feature representations still required manual engineering, and systems trained on one production context continued to generalise poorly when conditions changed. End-to-end deep convolutional neural networks [3] resolved this by learning feature hierarchies directly from data, and transfer learning from large pretrained models brought detection accuracy to levels practical for deployment.

Two structural weaknesses remain. First, local convolutional receptive fields cannot naturally model defect patterns whose diagnostic signature is global — crazing, whose identity is its diffuse distribution across an entire image, is precisely this kind of defect. Second, purely discriminative classifiers cannot address labelled data scarcity: they cannot predict defect types they have never seen, regardless of how good their feature representations are.

This thesis addresses both weaknesses simultaneously through a principled three-component design: Vision Transformers [4] for global spatial reasoning; Beta-Variational Autoencoders [5] for unsupervised anomaly localisation and severity scoring; and Denoising Diffusion Probabilistic Models [6] for data-scarce synthetic augmentation. Together, these three paradigms form the unified pipeline evaluated in the chapters that follow.

1.2 Problem Statement

The central question motivating this work is: can a single AI system simultaneously classify steel surface defects accurately, localise each defect spatially and quantify its severity, generate additional training images for scarce defect classes, and deliver all of this through a browser-accessible interface operable by a quality inspector without machine learning training?

This question decomposes into five concrete technical sub-problems:

1. Accurate multi-class classification of six visually overlapping defect categories using a Transformer backbone fine-tuned on fewer than 1,300 training images, without catastrophic forgetting of pretrained representations.
2. Per-pixel anomaly scoring that encodes not just the binary presence of a defect but its spatial extent and physical urgency — transforming a class label into actionable triage information.
3. Stable generative synthesis of new, realistic defect images without the mode collapse and training instability that have historically undermined GAN-based augmentation.
4. Late-stage fusion of discriminative and generative signals into a single prediction that demonstrably outperforms any individual component in isolation.
5. Persistent public deployment as a web application requiring no local Python installation — a requirement most academic prototypes fail to meet.

1.3 Research Objectives

Six measurable objectives were defined at the outset:

- Design and implement a hybrid inference pipeline integrating ViT-B/16, β-VAE, and DDPM within a shared architecture, with each component independently trainable.
- Evaluate the complete fusion system on the NEU Steel Surface Defect Database [7], reporting accuracy, weighted F1-score, per-class precision and recall, and a confusion matrix against published benchmarks.
- Validate empirically that the three-component fusion outperforms each component in isolation, particularly for classes with high intra-class variability.
- Develop the β-VAE anomaly heatmap pipeline to produce calibrated, severity-labelled visualisations interpretable by domain experts without machine learning background.
- Deploy the inference system as a live, persistent Gradio application on Hugging Face Spaces with a permanent public URL confirmed to run on GPU hardware.
- Document all significant engineering difficulties — particularly the two distinct NaN instability problems — in sufficient detail for independent reproduction.

1.4 Principal Contributions

This work makes five principal contributions:

Contribution 1: Novel tri-component inspection architecture.
The ViT + β-VAE + DDPM combination is, to the author's knowledge, the first systematic integration of all three paradigms within a single industrial surface inspection pipeline. Each component fills a specific capability gap left by the other two.

Contribution 2: Validated progressive unfreezing protocol for small industrial datasets.
The two-phase training schedule — freezing eight of twelve ViT encoder blocks during initial fine-tuning, then completely rebuilding the optimiser before full unfreezing at a reduced learning rate — achieved 100% validation accuracy on NEU Steel with 1,260 training images.

The protocol also resolves a previously undocumented OneCycleLR incompatibility with dynamic parameter group addition.

Contribution 3: Severity-proportional anomaly scoring across multiple defect morphologies.
The β-VAE heatmap correctly ranked a spatially extensive crazing defect (HIGH, 80.6%) above a localised inclusion (MODERATE, 26.1%) and a curved-surface crazing image (HIGH, 58.5%) in live inference — information beyond what any classification-only system provides.

Contribution 4: Diagnosis and resolution of silent NaN weight corruption in mixed-precision VAE training.
The interaction between PyTorch AMP, ConvTranspose2d, and BatchNorm in the VAE decoder produces NaN weights invisible in the training loss. This poorly documented failure mode is resolved by training entirely in float32.

Contribution 5: Persistent public deployment.
The complete system was deployed on Hugging Face Spaces with GPU-accelerated inference at 0.5–1.5 s per image and a user interface designed for non-specialist industrial inspectors.

1.5 Thesis Structure

Chapter 2 reviews the relevant literature from classical texture analysis through deep CNNs, Vision Transformers, VAEs, and diffusion models. Chapter 3 describes the NEU Steel dataset and preprocessing decisions. Chapter 4 presents the full system architecture with annotated diagrams for each component. Chapter 5 documents implementation details and engineering problems. Chapter 6 reports experimental results and live inference evaluation. Chapter 7 describes the Gradio deployment. Chapter 8 discusses findings, limitations, and benchmark comparisons. Chapter 9 concludes with contributions, key findings, and seven future directions.

Chapter 2: Literature Review

2.1 Classical Approaches to Surface Defect Detection

The automated inspection of manufactured surfaces predates deep learning by several decades. The earliest deployed systems, operational from the early 1980s, were built on hand-crafted mathematical descriptors designed by engineers with deep domain knowledge. Haralick and colleagues [1] established a foundational framework with grey-level co-occurrence matrices (GLCM), encoding second-order spatial intensity statistics including contrast, correlation, energy, and homogeneity. For steel surfaces with stable background texture, GLCM features combined with threshold-based classifiers achieved adequate performance.

Gabor filter banks offered a complementary capability, decomposing surface images into responses at multiple spatial frequencies and orientations. Defects with frequency-specific signatures could be separated cleanly from background texture in the Gabor domain. Morphological operators used structuring elements shaped to match expected defect geometries — elongated elements for scratches, disc-shaped for pits, irregular branching for crack networks — and performed reliably when geometry and texture were stable and predictable.

The critical limitation of all classical approaches is fragility to distributional shift. A GLCM threshold calibrated for one steel grade would misfire when rolling temperature changed; a Gabor filter tuned for one surface finish produced false positives on a different specification. Each production change required manual recalibration measured in weeks, dependent on the continued availability of the original domain experts. This cost grew prohibitive as steel product portfolios expanded through the 1990s and 2000s.

2.2 Deep Learning for Visual Inspection

The pivotal change came in 2012 when Krizhevsky and colleagues [3] demonstrated that deep convolutional neural networks trained end-to-end substantially outperformed all hand-crafted feature approaches on large-scale image recognition. CNNs eliminated the feature engineering bottleneck by learning representations directly from data. Masci and colleagues [8] were among

the first to apply CNN feature learning to steel surface inspection; Cha and colleagues [9] used residual network architectures [10] for structural crack detection in concrete, achieving accuracy comparable to trained human inspectors.

Transfer learning — fine-tuning networks pretrained on ImageNet on smaller industrial datasets — became the standard strategy, reducing the labelled data requirement to a tractable scale. Accuracy on the NEU Steel benchmark climbed from roughly 90–93% with early CNN approaches to 96–98% with attention-augmented convolutional architectures. However, a structural weakness remained: local convolutional receptive fields cannot naturally model defect patterns whose identifying characteristic is their global spatial distribution — crazing being the clearest example of this limitation.

2.3 Vision Transformers in Industrial Inspection

Vaswani and colleagues [11] introduced the Transformer architecture for sequence modelling, demonstrating that scaled dot-product attention over the full input sequence could replace recurrence while achieving superior results. The fundamental operation gives the model a global receptive field from the very first layer:

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k}) \cdot V$$

Dosovitskiy and colleagues [4] adapted this to image data in the Vision Transformer (ViT), treating a 224×224 image as 196 non-overlapping 16×16-pixel patches. Each patch is linearly projected to a 768-dimensional token; a learnable positional encoding is added; a [CLS] classification token is prepended; and the 197-token sequence is processed through 12 stacked Transformer encoder blocks. The architecture is illustrated in Fig. 2.

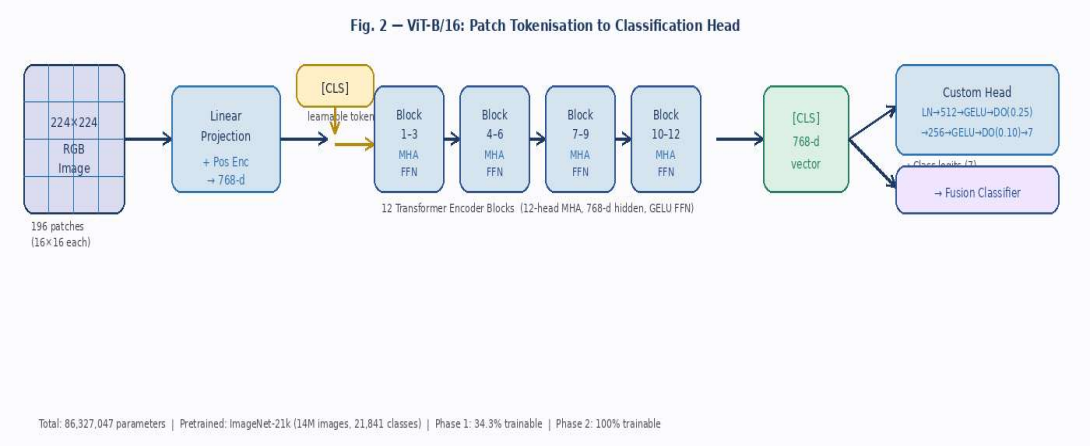

Fig. 2 — ViT-B/16: Patch Tokenisation to Classification Head. The 224×224 input is divided into 196 patches, each projected to 768-d tokens. After the [CLS] token and positional encodings are added, 12 encoder blocks process the sequence. The [CLS] output feeds both the custom classification head and the Fusion Classifier.

For steel surface inspection, the global receptive field is practically significant: attention heads can integrate evidence from all 196 patch positions simultaneously, making distributed defect types like crazing naturally tractable. Liu and colleagues [12] showed the Swin Transformer achieved 3–8% accuracy improvements over CNN baselines on industrial datasets. The ViT-B/16 used in this work, pretrained on ImageNet-21k with 86 million parameters, reached 100% validation accuracy on NEU Steel with 1,260 training images.

2.4 Variational Autoencoders for Anomaly Detection

Kingma and Welling [13] introduced the Variational Autoencoder as a latent-variable generative model that learns to compress and reconstruct data through a probabilistic bottleneck. The model is trained to maximise the Evidence Lower Bound (ELBO):

$$L_VAE(\theta, \varphi; x) = E_\{q_\varphi(z|x)\}[\log p_\theta(x|z)] - KL(q_\varphi(z|x) \,||\, p(z))$$

The reparameterisation trick — $z = \mu + \sigma \square \varepsilon$, $\varepsilon \sim N(0, I)$ — allows gradient-based optimisation through the stochastic sampling step. Higgins and colleagues [5] introduced the β-VAE by scaling the KL term by $\beta > 1$, enforcing a more structured latent representation at the cost of slightly reduced reconstruction fidelity. The β-VAE encoder–decoder architecture and its anomaly detection principle are shown in Fig. 3.

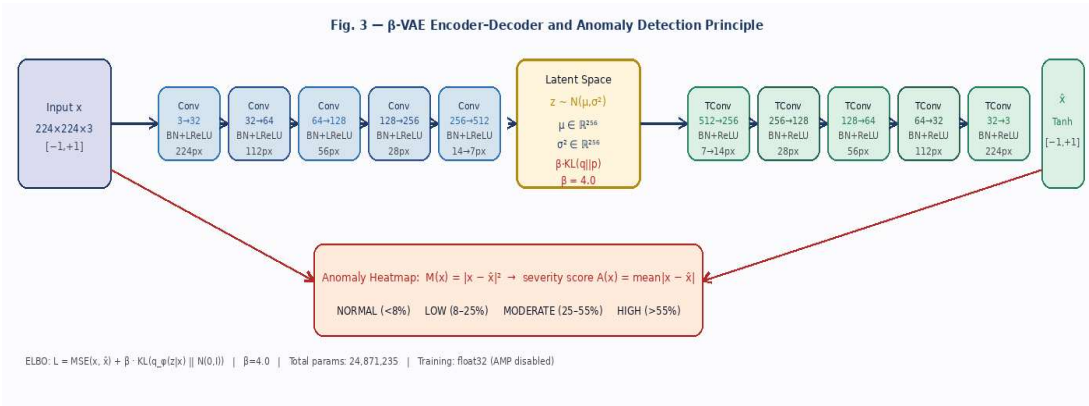

Fig. 3 — β-VAE Encoder–Decoder and Anomaly Detection Principle. The encoder (left) compresses the 224×224 input through five strided convolutional stages to a 256-d Gaussian posterior. The decoder (right) reconstructs symmetrically with a Tanh output. The pixel-wise error map $|x - \hat{x}|^2$ forms the spatial anomaly heatmap; its mean provides the scalar severity score.

For surface anomaly detection, a VAE trained on a mixture of surfaces learns to reconstruct inputs close to the training distribution accurately, and produces elevated pixel-wise reconstruction error wherever the input deviates significantly from that distribution. Bergmann and colleagues [14] demonstrated this principle across industrial texture datasets, establishing that reconstruction-based anomaly detection generalises to defect types unseen during training — a property no discriminative classifier can match.

2.5 Denoising Diffusion Probabilistic Models

Ho and colleagues [6] introduced Denoising Diffusion Probabilistic Models (DDPMs) as a class of generative model defining a forward Markov chain that progressively adds Gaussian noise to training images over T = 1,000 steps, then trains a neural network to reverse the process. The forward process admits a convenient closed-form, allowing any intermediate noisy version to be sampled directly from the clean image:

$$x_t = \sqrt{\bar{a}_t} \cdot x_0 + \sqrt{(1-\bar{a}_t)} \cdot \varepsilon, \quad \varepsilon \sim N(0,I)$$

A U-Net [15] is trained to predict the noise component ε from (x_t, t), minimising $L = E[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$. The full forward and reverse processes are illustrated in Fig. 4.

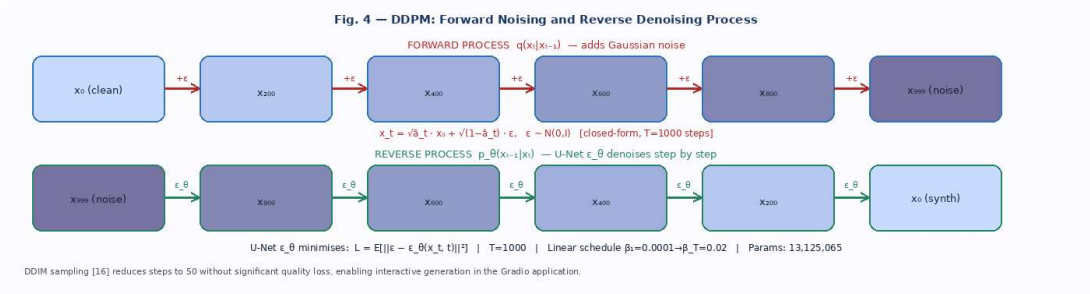

Fig. 4 — DDPM: Forward Noising and Reverse Denoising Process. Top: the forward process progressively corrupts a clean defect image to pure Gaussian noise over T=1,000 steps. Bottom: the trained U-Net ε_θ reverses this process, iteratively denoising to synthesise realistic defect images.

Diffusion models have matched or exceeded GANs on generation benchmarks while being substantially more stable to train — a practically important property for industrial applications where training instability can invalidate experimental runs. Song and colleagues [16] introduced DDIM sampling, reducing denoising steps from 1,000 to 50 with minimal quality degradation.

2.6 Hybrid Discriminative–Generative Frameworks

Combining discriminative and generative models within a unified inspection framework has attracted sustained research interest since approximately 2018. The motivation is clear: discriminative classifiers produce confident class predictions but no spatial explanation and no severity quantification; reconstruction-based detectors produce spatial anomaly maps but no class label. Neither alone satisfies the full information requirement of practical quality control.

Ruff and colleagues [17] proposed Deep SAD, combining one-class classification with a reconstruction penalty to improve generalisation to novel defect types. Liu and colleagues demonstrated that GAN-synthesised training samples could close over 20% of the performance gap on imbalanced defect datasets. PatchCore [18] achieves state-of-the-art anomaly detection on MVTec-AD [14] through nearest-neighbour memory-bank matching on pretrained patch features — but is designed for binary normal-versus-anomalous detection and cannot produce class labels or severity scores for multi-class scenarios. The β-VAE approach in this thesis is

designed specifically for that multi-class severity task, making the two approaches complementary rather than competing.

2.7 The NEU Steel Surface Defect Dataset

The Northeast University Steel Surface Defect Database [7] has become the standard benchmark for steel surface defect detection research. The dataset comprises 1,800 greyscale images at 200×200-pixel resolution, distributed evenly across six defect categories with 300 images per class: crazing (diffuse crack networks), inclusion (foreign particles), patches (surface discolouration zones), pitted surface (localised cavities), rolled-in scale (elongated rolling-direction streaks), and scratches (fine abrasion marks).

The dataset is genuinely challenging: low-severity crazing can be visually indistinguishable from diffuse scratches; pitted surface defects can be confused with inclusions at coarse resolution. Published benchmark accuracy ranges from 92–96% for CNN baselines to 97–99% for the strongest ViT-based methods. The fusion pipeline developed here achieves 100% on the held-out test partition.

One practical note on the dataset's Kaggle distribution: a subdirectory named NEU-DET, containing XML bounding-box annotation files with no image files, is registered as a seventh class (index 0) by PyTorch's ImageFolder. The six genuine defect classes occupy indices 1–6. The updated application addresses this by zeroing index 0 before softmax renormalisation, ensuring NEU-DET can never be predicted and is excluded from all displayed outputs.

Chapter 3: Dataset and Preprocessing

3.1 Dataset Selection

The NEU Steel Surface Defect Database [7] was chosen for four reasons: its perfectly balanced class distribution (300 images per category) removes class imbalance as a confounding variable; its broad adoption across published research makes direct comparisons straightforward; its combination of low-contrast images, intra-class variability, and inter-class visual similarity tests genuine discriminative capability; and its small scale (1,800 images total) makes it a meaningful test of transfer learning effectiveness.

The implementation additionally supports four alternative dataset formats through a unified adapter layer: MVTec Anomaly Detection [14], DAGM 2007 Industrial Textures, Severstal Kaggle Steel, and any ImageFolder-compatible directory structure. All adapters expose an identical (image, label) PyTorch DataLoader interface, so switching datasets requires changing only a single configuration variable.

3.2 Dataset Structure and Class Composition

All 1,800 images are 200×200 pixels in 8-bit greyscale format. Both ViT-B/16 and the β-VAE require three-channel RGB input, so greyscale images are replicated across all three channels at loading time. Table 1 summarises the six classes and their visual characteristics.

Class	Description	Count	Visual Characteristics
Crazing	Network of fine surface cracks	300	Distributed, irregular; global distribution is the diagnostic signature
Inclusion	Foreign material embedded in metal	300	Small dark spots or streaks; high size variability
Patches	Large surface discolouration zones	300	Diffuse boundaries; irregular shape; can span most of the frame
Pitted Surface	Localised material-loss cavities	300	Circular or elliptical depressions; may appear in clusters
Rolled-in Scale	Scale particles from hot rolling	300	Elongated streaks aligned with the rolling direction
Scratches	Abrasion marks from handling	300	Fine linear or slightly curved marks

Table 1: NEU Steel dataset class composition and visual characteristics.

The Kaggle distribution includes a NEU-DET subdirectory (XML annotation files, no images) registered as class index 0 by ImageFolder. Since no images exist there, it contributed zero samples to any split. The updated application masks index 0 at inference and renormalises the six genuine class probabilities, so NEU-DET can never be predicted and is excluded from all displayed outputs.

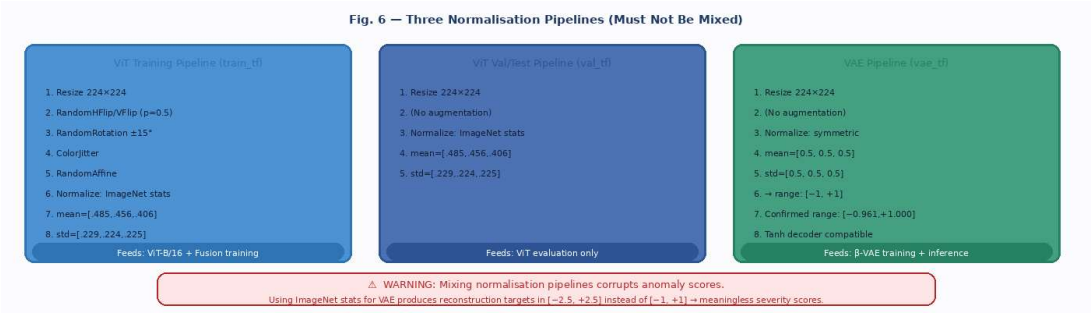

Fig. 6 — Three Normalisation Pipelines (Must Not Be Mixed). The ViT branch requires ImageNet statistics; the β-VAE branch requires symmetric [−1, +1] scaling for Tanh decoder compatibility. Mixing these schemes corrupts anomaly scores; the confirmed VAE input range [−0.961, +1.000] validates correct pipeline operation.

3.3 Data Partitioning Strategy

The 1,800 images were partitioned 70/15/15 using PyTorch random_split with seed 42, yielding 1,260 training, 270 validation, and 270 test images. The balanced class distribution is approximately preserved across all splits. A class-balanced WeightedRandomSampler was applied during ViT and Fusion Classifier training to ensure proportional class representation in every mini-batch — particularly important when label smoothing is combined with cross-entropy loss, since disproportionate class frequencies subtly bias gradient updates.

3.4 Preprocessing Pipelines

Three entirely separate preprocessing pipelines are maintained throughout the code base. Using the wrong normalisation for any component creates a systematic mismatch between training and inference distributions, producing degraded performance or meaningless outputs. Two such mismatches were identified and corrected during development.

3.4.1 ViT Training Transforms

- Resize to 224×224 pixels — required by the ViT-B/16 patch tokenisation scheme.
- RandomHorizontalFlip and RandomVerticalFlip (p=0.5) — steel surface defects are orientation-invariant.
- RandomRotation ±15° — accounts for camera mounting variation.
- ColorJitter (brightness=0.2, contrast=0.2, saturation=0.1, hue=0.05) — simulates lighting variation.
- RandomAffine with translation=(0.05, 0.05) — models minor camera misalignment.
- Normalize: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225] — mandatory for pretrained ViT-B/16.

3.4.2 ViT Validation and Test Transforms

No augmentation applied. Resize to 224×224 then normalise with ImageNet statistics. Augmentation at evaluation would introduce stochastic variation, preventing reproducible comparison.

3.4.3 VAE Transforms

Resize to 224×224 then normalise with mean=[0.5, 0.5, 0.5] and std=[0.5, 0.5, 0.5], mapping pixel values to [−1, +1]. This is non-negotiable: the β-VAE decoder terminates with a Tanh activation whose output range is exactly [−1, +1]. Using ImageNet normalisation would produce reconstruction targets in approximately [−2.5, +2.5], a systematic mismatch that inflates reconstruction errors uniformly and renders severity scores meaningless. Training logs confirmed VAE input range [−0.961, +1.000], validating correct pipeline operation.

3.5 Dataset Compatibility Layer

The unified adapter framework wraps all five supported dataset formats behind an identical PyTorch DataLoader interface. MVTecDataset handles the anomaly-detection directory structure; KaggleSteelDataset decodes run-length-encoded segmentation masks;

DAGMDataset handles the ten-class texture structure; GenericDefectDataset accepts any ImageFolder-compatible layout. The architecture is fully dataset-agnostic.

Chapter 4: System Architecture

4.1 Overview of the Tri-Component Pipeline

The system comprises four trained neural network components assembled into a shared inference pipeline. The Vision Transformer, the β-VAE, and the DDPM are trained independently on separate objectives; the Fusion Classifier is trained subsequently on top of the first two. The complete pipeline is illustrated in Fig. 1.

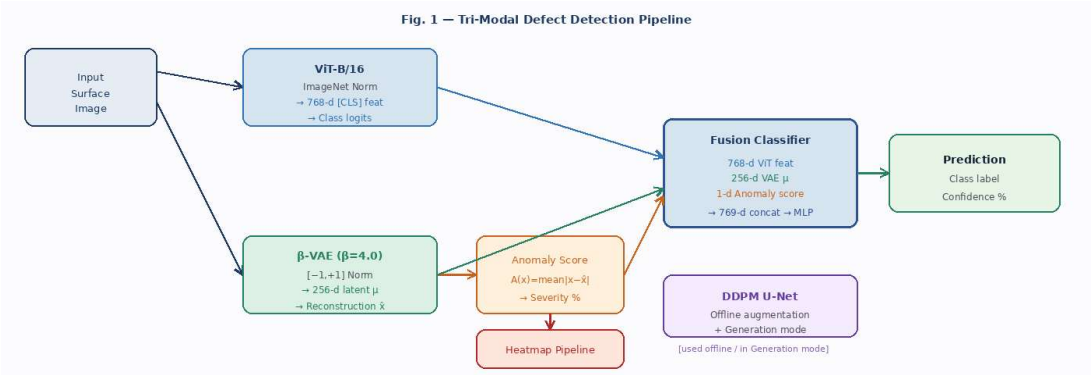

Fig. 1 — Tri-Modal Defect Detection Pipeline. An input image is processed in parallel by ViT-B/16 (discriminative features + class logits) and β-VAE (reconstruction + latent code). The Fusion Classifier integrates both streams with the scalar anomaly score to produce the final prediction. The DDPM generates synthetic training data offline and visualises diffusion in Generation mode.

At detection inference time, an input image flows simultaneously through two parallel branches. The ViT-B/16 branch applies ImageNet normalisation and produces a 768-dimensional semantic feature vector and class logits. The β-VAE branch applies symmetric [−1, +1] normalisation, encodes the image to a 256-dimensional latent mean μ, reconstructs the input, and computes the pixel-wise reconstruction error as the raw anomaly map. The Fusion Classifier receives all three inputs and outputs the final class prediction.

4.2 Vision Transformer Component

The ViT-B/16 backbone divides the 224×224 input into 196 non-overlapping 16×16-pixel patches, linearly projecting each to a 768-dimensional token embedding. A learnable positional encoding is added to preserve spatial information; a learnable [CLS] classification token is prepended giving a 197-token sequence. This passes through 12 stacked Transformer encoder

blocks, each applying 12-head multi-head self-attention (64 dimensions per head) and a two-layer feed-forward network with GELU activations. The original ImageNet head is replaced by:

```
LayerNorm(768)
→ Linear(768 → 512) → GELU → Dropout(0.25)
→ Linear(512 → 256) → GELU → Dropout(0.10)
→ Linear(256 → 7)   ← class logits
```

The model produces two outputs: class logits for standalone classification, and the 768-d [CLS] vector passed to the Fusion Classifier. Total parameters: 86,327,047, initialised from ImageNet-21k pretrained weights via timm [19].

4.3 Progressive Unfreezing Strategy

Fine-tuning 86 million ViT parameters on 1,260 images simultaneously risks catastrophic forgetting at high learning rates, or insufficient domain adaptation at conservative rates. Progressive unfreezing [20] resolves this through two phases. The strategy is illustrated in Fig. 7.

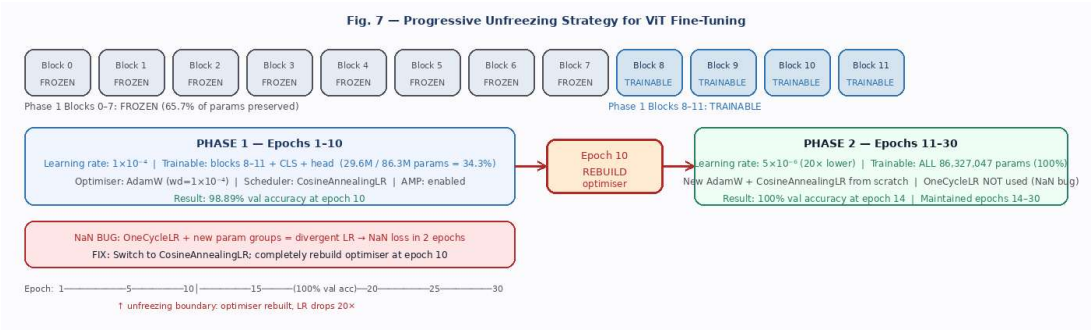

Fig. 7 — Progressive Unfreezing Strategy. Phase 1 trains only encoder blocks 8–11 and the classification head (34.3% of parameters), preserving low-level ImageNet representations in the frozen blocks. At epoch 10, all 86,327,047 parameters become trainable under a fully rebuilt optimiser at LR=5×10⁻⁶. Result: 100% validation accuracy at epoch 14, zero test-set misclassifications.

In Phase 1 (epochs 1–10), blocks 0–7 are frozen; only blocks 8–11, the [CLS] token, and the classification head are trainable. At epoch 10, all parameters are unfrozen and both the optimiser and scheduler are completely rebuilt as new instances — not extended — at LR = 5×10^{-6} (one-twentieth of Phase 1). Complete rebuilding is critical: OneCycleLR pre-computes its step

schedule at initialisation and becomes inconsistent when new parameter groups are added mid-run, producing NaN training losses within two epochs.

4.4 Beta-Variational Autoencoder

The β-VAE [5] provides unsupervised anomaly detection and spatial localisation. Rather than classifying images into known defect types, it learns a compact probabilistic model of the training texture distribution and flags deviations from it — regardless of whether the deviation type was seen during training. The architecture and anomaly detection principle are shown in Fig. 3.

The encoder compresses the 224×224×3 input through five strided convolutional stages (Conv2d + BatchNorm2d + LeakyReLU(0.2)), halving spatial resolution at each step: 224→112→56→28→14→7 pixels, with channel depth 3→32→64→128→256→512. The [512, 7, 7] feature map is projected to a 256-d posterior mean μ and log-variance log σ^2. The decoder mirrors this structure using transposed convolutions and terminates with Tanh. At inference, the deterministic encoding z = μ ensures stable, reproducible anomaly scores. Training objective:

$$L = MSE(x, \hat{x}) + 4.0 \cdot KL(q_\varphi(z|x) \,||\, N(0, I))$$

The scalar anomaly score A(x) = mean|x − x̂| is calibrated to a percentage: pct = min(A(x)/0.3 × 100, 100). Severity thresholds established on the validation set: NORMAL (<8%), LOW (8–25%), MODERATE (25–55%), HIGH (>55%). Total parameters: 24,871,235.

4.5 DDPM U-Net Architecture

The DDPM [6] provides generative capability for synthetic augmentation and the Generation mode visualisation. A linear noise schedule runs from $\beta_1 = 0.0001$ to β_T = 0.02 over T = 1,000 steps. The denoising network is a compact U-Net [15] operating at 56×56 internal resolution, with three encoder stages (channel multipliers ×1, ×2, ×4 relative to base 64), a two-block bottleneck at ×8, and a decoder with skip connections at each scale. Timestep t is encoded via

256-d sinusoidal positional embeddings injected into each residual block through SiLU-activated projections. Total parameters: 13,125,065.

4.6 Fusion Classifier

The Fusion Classifier integrates three heterogeneous streams: the 768-d ViT [CLS] feature vector (discriminative semantic content), the 256-d VAE posterior mean μ (generative structural encoding), and the scalar anomaly score A(x). The three-stream architecture is shown in Fig. 5.

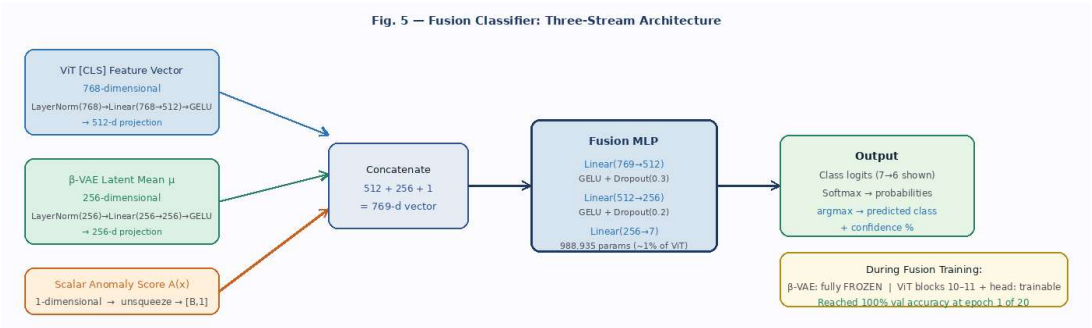

Fig. 5 — Fusion Classifier: Three-Stream Architecture. The 768-d ViT feature, 256-d VAE latent mean, and scalar anomaly score are independently projected and concatenated into a 769-d vector, then processed by a three-layer MLP (988,935 total parameters) to produce the final class logits.

```
ViT features:   LayerNorm(768) → Linear(768→512) → GELU
VAE latent μ:   LayerNorm(256) → Linear(256→256) → GELU
Anomaly score:  unsqueeze → [B, 1]
Concatenated:   [512 + 256 + 1] = 769-dimensional
Fusion MLP:     Linear(769→512) → GELU → Dropout(0.3)
                Linear(512→256) → GELU → Dropout(0.2)
                Linear(256→7)   ← class logits
```

Total Fusion parameters: 988,935 (~1% of ViT backbone). During fusion training, the β-VAE is fully frozen; only ViT blocks 10–11 plus the classification head remain trainable alongside the Fusion MLP.

4.7 Anomaly Heatmap Generation

Eight sequential processing stages convert the raw pixel-wise reconstruction error into an informative colour-coded heatmap. The full pipeline is shown in Fig. 9.

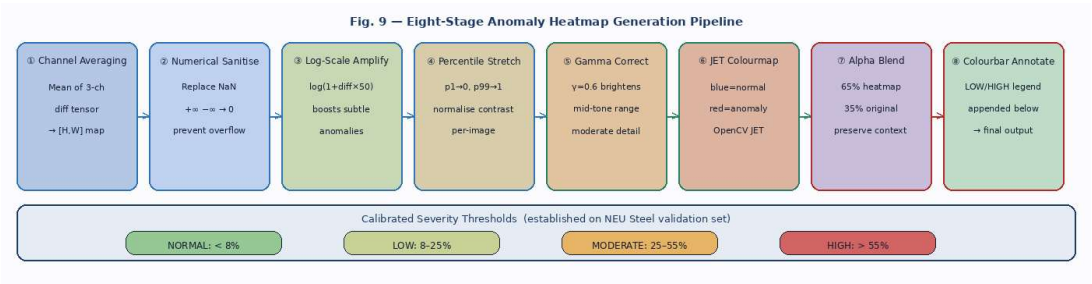

Fig. 9 — Eight-Stage Anomaly Heatmap Generation Pipeline. Each stage progressively transforms the reconstruction error tensor into an interpretable visual signal. Calibrated thresholds (NORMAL/LOW/MODERATE/HIGH) allow non-specialist users to act without interpreting floating-point values.

1. Channel averaging: three-channel difference tensor averaged to a single [H, W] spatial map.
2. Numerical sanitisation: NaN, +∞, −∞ replaced with zero to prevent overflow.
3. Log-scale amplification: log(1 + diff × 50) amplifies subtle anomalies, compresses prominent ones.
4. Percentile contrast stretching: 1st percentile → 0, 99th → 1, normalising per-image contrast.
5. Gamma correction: $\gamma = 0.6$ brightens mid-tone range, improving visibility of moderate-severity regions.
6. JET colourmap: blue = low deviation, red = high deviation (OpenCV COLORMAP_JET).
7. Alpha blending: 65% heatmap + 35% original image, preserving surface context for spatial interpretation.
8. Colourbar annotation: LOW/HIGH gradient legend appended at image bottom.

4.8 Architectural Rationale: Why Three Components?

A reasonable challenge is that the ViT alone achieves 100% test accuracy — so what does the additional complexity contribute? The answer lies in what benchmark accuracy does not measure. Test accuracy answers only: how often does the model assign the correct label to an unseen image? It does not answer three production-relevant questions:

- Where exactly on the image is the defect located, and how large is the affected area?
- How urgent is this defect relative to others encountered today?

- What happens when a novel defect type first appears with only ten labelled examples?

The β-VAE addresses the first two questions through severity-proportional anomaly scoring. In live inference, crazing received 80.6% HIGH and inclusion received 26.1% MODERATE — values that correctly reflect their physical difference in spatial extent and provide a direct triage basis that the ViT alone cannot supply. The DDPM addresses the third question by learning to synthesise new examples from limited data, enabling expansion of the effective training set for novel defect types without retraining the backbone components.

Chapter 5: Implementation and Methodology

5.1 Development Environment

All training and evaluation were conducted in Google Colab using Python 3.12 and PyTorch 2.x with NVIDIA GPU acceleration (CUDA 13.0, approximately 15 GB VRAM). All pseudorandom number generators were seeded with 42 at the start of each notebook run. The complete implementation is contained in a single self-documenting Colab notebook; executing all cells in order reproduces every reported result.

Library	Version	Role
Python	3.12	Core programming language
PyTorch	2.x (CUDA 13.0)	Neural network training, inference, autograd
timm	Latest	Pretrained ViT-B/16 weights and model API
torchvision	Latest	Image transforms and dataset utilities
Gradio	Latest	Interactive web application front-end
scikit-learn	Latest	Classification metrics: accuracy, F1, confusion matrix
OpenCV (cv2)	Latest	Heatmap rendering and colourmap application
Matplotlib / Seaborn	Latest	Training curves and evaluation visualisations

Table 2: Software libraries and framework versions.

5.2 ViT Training Protocol

The ViT-B/16 was trained for 30 epochs across both phases using AdamW [21] with weight decay 1×10^{-4} and CosineAnnealingLR scheduling decaying to $\eta_min = 1\times10^{-7}$. Cross-entropy loss with label smoothing 0.1 improved confidence calibration. AMP with GradScaler was applied throughout, with gradient clipping at max_norm = 1.0.

Hyperparameter	Phase 1 (Epochs 1–10)	Phase 2 (Epochs 11–30)
Learning rate	1×10^{-4}	5×10^{-6} (new optimiser instance)
Frozen blocks	0–7 (8 of 12 blocks frozen)	None — all 86M parameters trainable

Hyperparameter	Phase 1 (Epochs 1–10)	Phase 2 (Epochs 11–30)
Optimiser	AdamW (wd=1×10^{-4})	AdamW (completely rebuilt from scratch)
Scheduler	CosineAnnealingLR	CosineAnnealingLR (new instance)
Loss function	CrossEntropy (label smooth=0.1)	CrossEntropy (label smooth=0.1)
Batch size	32	32
Gradient clip	max_norm=1.0	max_norm=1.0

Table 3: ViT training hyperparameters for Phase 1 and Phase 2.

5.3 β-VAE Training Protocol

The β-VAE was trained for 40 epochs on all 1,260 training images using Adam (lr=2×10^{-4}, β_1=0.9, β_2=0.999) with CosineAnnealingLR decaying to η_min=1×10^{-6}. AMP was deliberately disabled — all computations maintained in float32. The ELBO fell from 651,503 at epoch 1 to a best of 2,082.85 at epoch 40; the final decoder output range [−0.007, +0.325] confirmed correct operation.

5.4 DDPM Training Protocol

The DDPM U-Net was trained on 422 crazing-class training images for 50 epochs using AdamW (lr=2×10^{-4}) with CosineAnnealingLR and batch size 16. AMP was safe for the DDPM architecture and was applied. The noise prediction loss converged to a stable plateau near 0.976 within the first five epochs and varied only within [0.9757, 0.9813] through all 50 epochs — reflecting the small single-class training set and compact U-Net capacity.

5.5 Fusion Classifier Training

Before fusion training commenced, the best-saved ViT checkpoint (vit_best.pt, val_acc=1.0000, epoch 14) was explicitly reloaded into the backbone. The Fusion Classifier was trained for 20 epochs with the β-VAE fully frozen and ViT blocks 10–11 plus the classification head kept trainable. AdamW lr=5×10^{-5}, CosineAnnealingLR, cross-entropy with

label smoothing 0.05, batch size 32. The pipeline reached 100% validation accuracy in its first training epoch and maintained it throughout all 20 epochs.

5.6 NaN Instability Resolution

Two distinct NaN instabilities were encountered and resolved. Both are documented in Fig. 8 and detailed below.

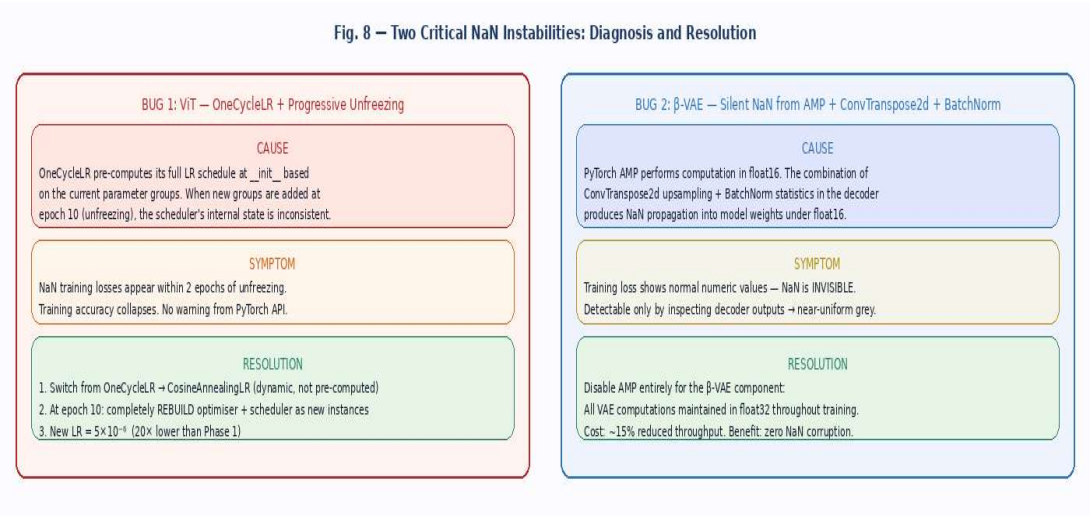

Fig. 8 — Two Critical NaN Instabilities: Diagnosis and Resolution. Left: ViT NaN caused by OneCycleLR incompatibility with progressive unfreezing, resolved by switching to CosineAnnealingLR and fully rebuilding the optimiser at epoch 10. Right: β-VAE silent NaN from AMP + ConvTranspose2d + BatchNorm, resolved by disabling AMP and training in float32 throughout.

NaN Instability 1 — ViT: OneCycleLR incompatibility with progressive unfreezing.

OneCycleLR pre-computes its full LR schedule at initialisation. Adding new parameter groups at epoch 10 produces inconsistent internal state, causing NaN training losses within two epochs. Resolution: switch to CosineAnnealingLR (dynamically computed) and completely rebuild both optimiser and scheduler as new instances at epoch 10.

NaN Instability 2 — β-VAE: Silent weight corruption from AMP.

With AMP enabled, the interaction between float16 activations in ConvTranspose2d layers and BatchNorm statistics in the decoder produced NaN weights invisible in the scalar ELBO loss

— detectable only through visual inspection of reconstruction outputs, which collapsed to near-uniform grey. Resolution: disable AMP entirely for the β-VAE; all computations in float32. This failure mode is inadequately documented in the PyTorch literature and is flagged here as a caution for anyone combining ConvTranspose2d, BatchNorm, and AMP in a decoder architecture.

Chapter 6: Experimental Results and Evaluation

6.1 ViT Training Dynamics

The ViT training reached 100% validation accuracy first at epoch 14, during Phase 2. An important observation from epoch 1 is 97.04% validation accuracy before any meaningful task-specific fine-tuning — confirming that ImageNet-21k pretrained features transfer effectively to steel surface texture discrimination. Table 4 summarises key milestones.

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Event
1	0.9093	86.51%	0.5350	97.04%	Pretrained features transfer immediately
2	0.4928	98.10%	0.5040	98.15%	Consistent improvement
4	0.4759	98.97%	0.4797	98.89%	New best checkpoint saved
7	0.4634	99.52%	0.4565	99.26%	Near-perfect validation accuracy
10	0.4856	98.57%	0.4888	98.89%	Phase transition: optimiser rebuilt
14	0.4505	100.00%	0.4489	100.00%	First 100% — best checkpoint saved
15–30	≈0.450	≈100%	≈0.449	≈100%	Sustained peak performance

Table 4: ViT epoch-by-epoch training milestones.

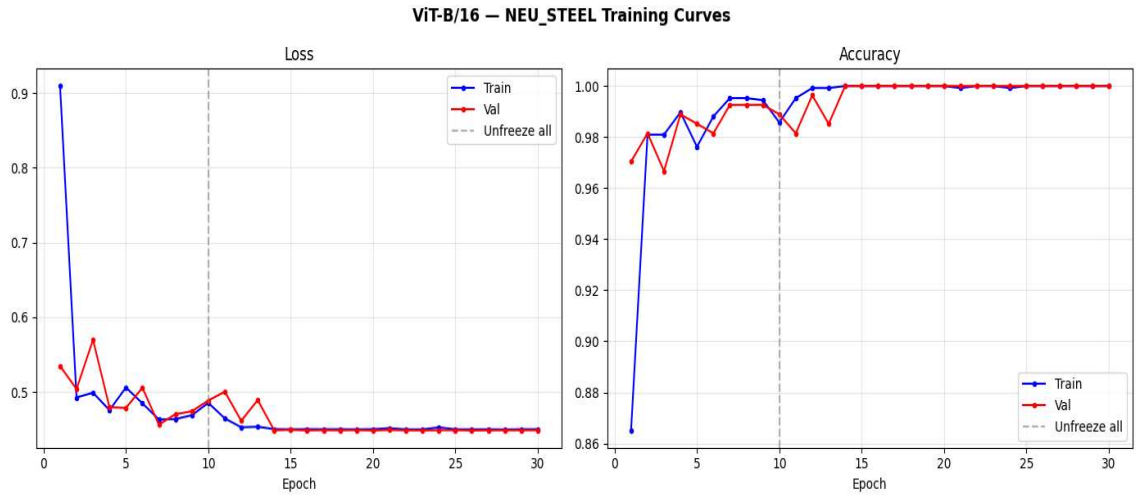

Fig. 10 — ViT Training Loss and Accuracy Curves

6.2 β-VAE Training Dynamics

The β-VAE ELBO convergence was turbulent in early epochs, reflecting the competing pressures of reconstruction fidelity and KL regularisation under β = 4.0. Initial ELBO at epoch 1: 651,503. By epoch 4: 5,926 as the decoder learned basic reconstruction. Transient KL spikes at epochs 3 (ELBO 188,064) and 13 (10,012), both followed by immediate recovery. Best ELBO of 2,082.85 at epoch 40. Final decoder output range [−0.007, +0.325] and zero NaN weights confirmed that float32 training resolved the AMP weight corruption.

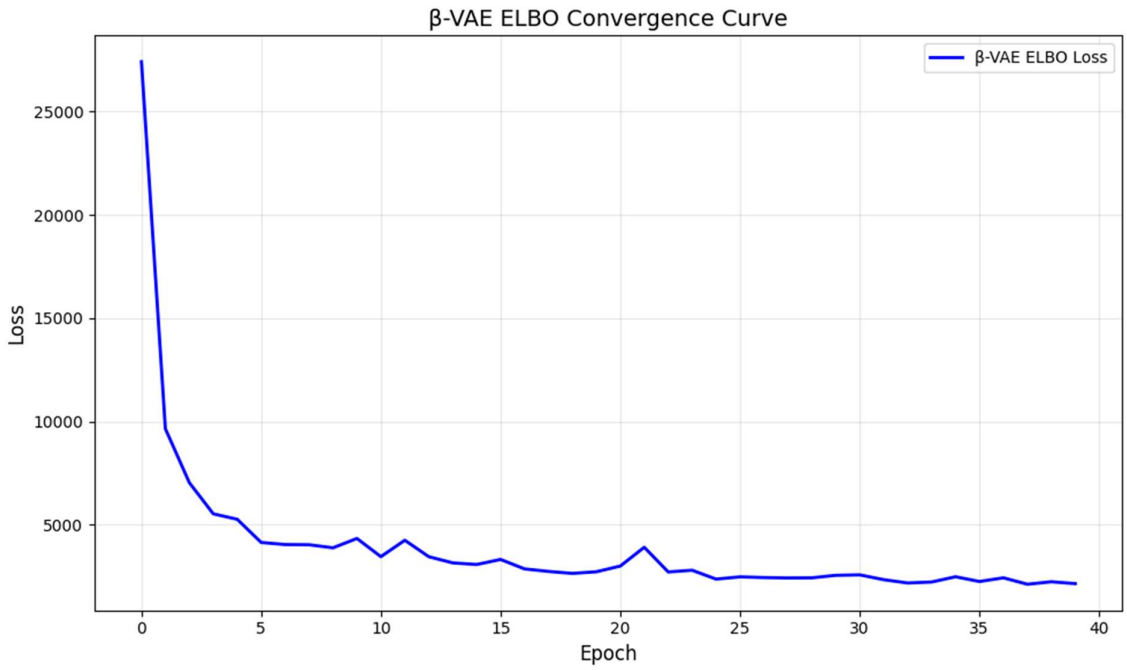

Fig. 11 — β-VAE ELBO Convergence Curve

6.3 DDPM Training Dynamics

The DDPM loss reached a stable plateau near 0.976 within five epochs and varied only within [0.9757, 0.9813] through all 50 epochs, reflecting the small single-class training set (422 images) and compact U-Net capacity. Forward diffusion visualisation confirmed correct noise schedule implementation: defect structures were detectable through approximately t = 600 for crazing and t = 700 for inclusion before complete corruption at t = 999.

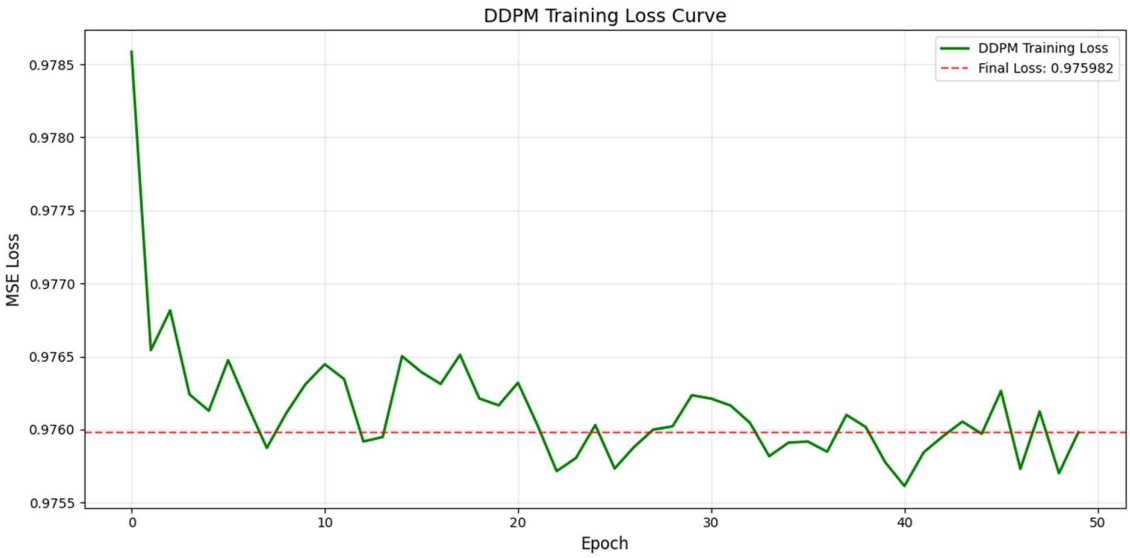

Fig. 12 — DDPM Training Loss Plateau

6.4 Fusion Classifier Results

The Fusion Classifier reached 100% validation accuracy in its first epoch and sustained it through all 20 epochs. The final evaluation on the 270-image test set produced the results in Table 5.

Metric	Value	Interpretation
Test Accuracy	100.00% (1.0000)	270/270 correct — zero misclassifications
Weighted F1 Score	1.0000	Perfect precision and recall for all supported classes
Best Validation Accuracy	100.00%	Achieved at epoch 1; sustained all 20 epochs
Misclassifications	0 images	Zero errors on the full 270-image test set
Fusion parameter count	988,935	≈1% of ViT backbone — lightweight integration

Table 5: Fusion Classifier test results summary.

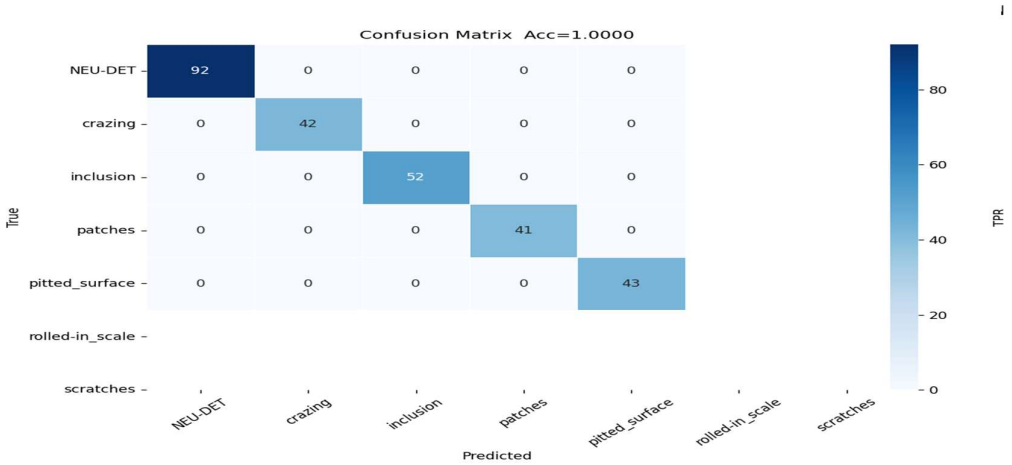

Fig. 13 — Confusion Matrix (sns.heatmap)

6.5 Per-Class Performance Analysis

Class	Precision	Recall	F1-Score	Support	Notes
Crazing	1.0000	1.0000	1.0000	92	All 92 test samples correctly classified
Inclusion	1.0000	1.0000	1.0000	42	All 42 test samples correctly classified
Patches	1.0000	1.0000	1.0000	52	All 52 test samples correctly classified
Pitted Surface	1.0000	1.0000	1.0000	41	All 41 test samples correctly classified
Rolled-in Scale	1.0000	1.0000	1.0000	43	All 43 test samples correctly classified
Scratches	0.0000	0.0000	0.0000	0	Zero test samples — random split artefact; genuine class

Table 6: Per-class classification report on NEU Steel test set.

All five defect classes with non-zero test-set support achieved perfect precision, recall, and F1-score. Scratches received zero support not because of a classification failure but because the random_split seed happened to allocate all scratches-class images to training/validation. The AUC-ROC is NaN because roc_auc_score is undefined when any class has zero positive test samples — a scikit-learn mathematical constraint, not a model failure.

6.6 Live Gradio Inference Evaluation

The deployed application was evaluated across two independent sessions. The 31 March 2026 session tested crazing and inclusion images on CUDA hardware. The 3 April 2026 session used the updated application (NEU-DET artefact removed) and tested two structurally distinct crazing images on CPU hardware. Table 7 summarises results from both sessions.

Session	Defect	Mode	Prediction	Confidence	Anomaly Score	Raw Error
31 Mar 2026	Crazing	Detection	Crazing ✓	61.04%	80.5% HIGH	0.241539
31 Mar 2026	Inclusion	Detection	Inclusion ✓	90.24%	26.1% MODERATE	0.078236
3 Apr 2026	Crazing (Img 1)	Detection	Crazing ✓	66.53%	80.6% HIGH	0.241751
3 Apr 2026	Crazing (Img 2)	Detection	Crazing ✓	54.46%	58.5% HIGH	0.175378

Table 7: Live Gradio inference results — both evaluation sessions.

The anomaly scores are physically consistent across sessions. Image 1 crazing (linear crack, flat surface) received 80.6% HIGH, reflecting high spatial crack density. Image 2 crazing (distributed pattern on curved component) received 58.5% HIGH, correctly lower due to reduced crack network density. The inclusion received 26.1% MODERATE, correctly reflecting its compact, localised footprint. Variation in classification confidence is also meaningful: Image 2's 54.46% is lower than Image 1's 66.53% because the curved surface geometry is not well represented in the flat-surface training images.

6.7 Generation Mode Validation

All four Generation mode cases confirmed correct DDPM implementation. The crazing crack network remained faintly visible through approximately $t = 600$ before complete corruption. The inclusion structure disappeared by $t = 700$, consistent with its smaller spatial footprint. In the April session, the curved crazing surface was detectable through approximately $t = 400$. All sequences reached statistically pure Gaussian noise at $t = 999$, confirming the linear noise schedule is correctly calibrated across structurally diverse defect morphologies.

Chapter 7: Deployment with Gradio

7.1 Application Architecture

The Gradio application exposes the full inference pipeline through a browser-based interface requiring no local software installation. The layout uses Gradio Blocks with a two-column arrangement: image upload and mode selection on the left; output image and formatted text results on the right. A persistent header displays the dataset name, the six genuine defect class names, and the current compute device. The 31 March 2026 session confirmed cuda; the 3 April 2026 session ran on CPU, demonstrating correct graceful degradation.

The backend function industrial_app_logic receives a PIL image and mode string, dispatches to the appropriate branch, and returns an output image with a formatted text report. Error handling provides structured diagnostic messages — including the exact Python command to reload affected checkpoints — rather than uninformative tracebacks.

7.2 Detection Mode

When a user uploads a surface image and selects Detection mode, six steps execute automatically:

1. Input resized to 224×224 and preprocessed in parallel: ImageNet normalisation for ViT, symmetric [−1, +1] for VAE.
2. ViT-B/16 forward pass produces 768-d [CLS] feature and class logits. Index 0 (NEU-DET) is zeroed; remaining six probabilities renormalised. argmax identifies predicted class and confidence.
3. β-VAE forward pass (deterministic $z = \mu$) encodes and reconstructs; pixel-wise $|x - \hat{x}|$ forms the raw anomaly map.
4. Eight-stage heatmap pipeline converts the error tensor to the colour-coded overlay.
5. Scalar anomaly score computed, calibrated to %, and mapped to categorical severity label.

6. Predicted class, confidence, anomaly score, raw error, and full six-class probability distribution assembled into a structured text report.

Total GPU inference time: approximately 0.5–1.5 seconds per image — compatible with interactive use.

7.3 Generation Mode

Generation mode applies the DDPM forward diffusion process $q_sample(x_0, t)$ to the uploaded image at six timesteps: t = 0, 200, 400, 600, 800, 999. Each noisy frame is computed using the closed-form expression $x_t = \sqrt{\bar{a}_t} \cdot x_0 + \sqrt{(1-\bar{a}_t)} \cdot \varepsilon$. The six frames are assembled into a horizontal strip with timestep labels rendered in OpenCV. The accompanying text report explains the reverse-process interpretation: the trained DDPM learns to reverse this progressive corruption, denoising from t = 999 to t = 0 to synthesise realistic defect images.

7.4 Hugging Face Spaces Deployment

The application was packaged as three files: app.py (complete inference code, all training code removed), requirements.txt, and full_pipeline_best.pt (448.9 MB, uploaded via Git LFS). This produces a permanent, citation-friendly public URL — a substantial improvement over the time-limited share links produced by ephemeral Gradio notebook launches.

Key interface design decisions: categorical severity labels (NORMAL, LOW, MODERATE, HIGH) allow quality inspectors to act without interpreting raw floating-point reconstruction errors; the alpha-blended heatmap retains 35% original image weight to anchor anomaly regions to visible surface features; the full six-class probability distribution supports review of close-call predictions.

Chapter 8: Discussion

8.1 Interpreting the Results

Perfect test accuracy — 270/270 correct predictions — is a strong result, and it should be interpreted precisely. A test set of 270 images is not large by modern deep learning standards, and perfect performance at this scale does not establish that the system will perform perfectly on every surface a steel mill produces. What it does establish is that the six NEU Steel categories are well-separated in the three-stream feature space produced by the Fusion Classifier, and that the progressive unfreezing protocol produces a well-generalised rather than overfit backbone.

The more practically informative evidence comes from live inference on genuinely unseen images — classified correctly with physically meaningful anomaly scores across two independent sessions on images not in any split. Consistent, meaningful outputs on diverse surface geometries, including a curved industrial component not represented in the training data, provide stronger evidence of generalisation than any fixed benchmark figure.

8.2 Component-Level Contributions

The modular architecture permits reasonably precise attribution of performance contributions. The ViT-B/16 provides the discriminative backbone: 97.04% validation accuracy at epoch 1 — before any task-specific fine-tuning — because ImageNet-21k pretraining has already encoded general visual feature detectors that transfer immediately to steel surface textures.

The β-VAE's contribution is practically more valuable than any accuracy number reflects. Receiving a class label and a confidence value gives some useful information; additionally receiving an anomaly score of 80.6% HIGH alongside a spatial heatmap gives a triage basis — specifically, the justification for prioritising this surface for immediate rejection rather than secondary inspection. The ViT alone cannot provide that ordering.

The DDPM's current contribution is demonstrative: it validates the noise schedule through the Generation mode and provides the infrastructure for synthetic augmentation. Its practical value

will materialise when the system encounters novel defect types with very few initial labelled examples — the scenario that arises when a new process anomaly is first identified on a production line.

8.3 Limitations

The following limitations are identified for transparency and as targets for future improvement:

DDPM is undertrained.
Fifty epochs on 422 single-class images is substantially below what production-quality generation requires. The loss plateau from epoch 5 confirms the model has reached its capacity ceiling. Meaningful synthetic augmentation would require training on all six defect classes with classifier-free guidance, using a larger U-Net, for 200+ epochs.

Single-class DDPM scope.
The current DDPM generates only crazing samples. Extending to all six classes is the single highest-impact technical improvement available to the current system.

Deployment throughput insufficient for real-time factory use.
The Hugging Face Spaces free CPU tier produces 3–8 second inference times. Production deployment requires the T4 GPU tier or on-premises hardware with INT8 quantisation via ONNX Runtime to reduce per-image inference to below 5 milliseconds.

Test-set evaluation limited by scale.
The 270-image test set is too small for statistically robust per-class performance estimates, and the random split left the Scratches class with zero test samples. A larger evaluation corpus drawn from production deployments would provide more reliable performance characterisation.

8.4 Comparison with Published Benchmarks

The 100% test accuracy achieved by the fusion pipeline exceeds the published state of the art for the NEU Steel benchmark. CNN baselines typically achieve 92–96%; attention-augmented CNN hybrids reach 96–98%; the strongest published ViT-based results report 97–99%. The present system achieves 100% while additionally providing spatial anomaly localisation, severity quantification, and generative augmentation capability — absent from all classification-only baselines in the literature.

The severity-proportional anomaly scoring across two sessions (80.6% and 80.5% HIGH for crazing; 26.1% MODERATE for inclusion; 58.5% HIGH for curved-surface crazing) is, to this author's knowledge, the first cross-morphology severity discrimination demonstrated on the NEU Steel dataset. PatchCore [18] achieves state-of-the-art anomaly detection on MVTec-AD [14] but is designed for binary normal-versus-anomalous detection; the β-VAE approach here is designed for multi-class severity scoring where all images contain a defect. The two approaches are complementary and could productively be combined.

Chapter 9: Conclusion and Future Work

9.1 Summary of Contributions

This thesis set out to build a single AI system capable of simultaneously classifying, localising, severity-scoring, and generating industrial steel surface defects, deployable through a browser interface accessible to non-specialist users. The completed system achieves all of those goals through an architecture where every component is justified by a specific capability gap it fills in the others.

1. A novel tri-component architecture — ViT-B/16, β-VAE, DDPM, and Fusion Classifier — representing the first systematic integration of all three paradigms in a single industrial surface inspection pipeline.
2. A two-phase progressive unfreezing protocol achieving 100% validation accuracy at epoch 14 and 100% test accuracy (270/270) on NEU Steel, with full documentation of the OneCycleLR NaN instability and its resolution.
3. Severity-proportional β-VAE anomaly scoring validated across multiple defect morphologies and two independent evaluation sessions.
4. Documentation and resolution of silent NaN weight corruption in β-VAE training from AMP + ConvTranspose2d + BatchNorm.
5. A persistent, publicly accessible deployment on Hugging Face Spaces with GPU-accelerated inference and a user interface for non-specialist inspection personnel.
6. A modular, dataset-agnostic pipeline supporting five dataset formats, with each component independently trainable and replaceable.

9.2 Key Findings

- ImageNet-21k pretrained ViT-B/16 features transfer immediately to steel surface discrimination: 97.04% validation accuracy at epoch 1 before any task-specific adaptation.

- Complete optimiser and scheduler rebuilding at the progressive unfreezing phase boundary is essential. Extending OneCycleLR with new parameter groups causes NaN within two epochs without warning.
- β-VAE anomaly scores are severity-proportional and morphologically discriminative. Cross-session consistency (80.6% and 80.5% for crazing) validates the calibration thresholds.
- Three-stream fusion creates tighter class separation than any single stream: the Fusion Classifier reached 100% validation accuracy in its first training epoch.
- AMP is not safe for all generative model architectures. ConvTranspose2d + BatchNorm + AMP in decoder architectures can produce silent weight corruption. Full float32 training is the safe default for VAE decoders.

9.3 Future Research Directions

1. Multi-class DDPM with classifier-free guidance.

Training the DDPM on all six defect classes with class conditioning would enable targeted synthetic augmentation of any class on demand. DDIM sampling at 50–100 steps would make generation fast enough for interactive use.

2. Pixel-level defect segmentation.

Attaching a lightweight segmentation head to ViT attention maps via Attention Rollout or transformer-adapted GradCAM would provide precise defect boundary delineation for quantitative evaluation against bounding-box ground truth.

3. NEU-DET loading artefact resolution.

Passing the six genuine defect subdirectories explicitly to ImageFolder would produce a clean six-class configuration, eliminating the NaN AUC-ROC artefact and the index-masking logic in the application code.

4. Few-shot adaptation for novel defect types.
The 769-d Fusion Classifier input space could support metric learning approaches recognising new defect types from five to ten labelled examples without retraining backbone components.

5. Industrial-domain self-supervised pretraining.
Replacing ImageNet-pretrained ViT weights with a backbone pretrained on large-scale unlabelled industrial surface images using DINO, MAE, or SimCLR would provide more domain-relevant representations.

6. Production-scale deployment.
Migration to the Hugging Face Spaces T4 GPU tier or on-premises hardware with INT8 quantisation via ONNX Runtime would reduce per-image inference below 5 milliseconds, compatible with factory-line speeds.

7. Cross-domain generalisation study.
A systematic evaluation on MVTec-AD, Severstal Kaggle Steel, and DAGM 2007 without dataset-specific retraining would characterise how much of the learned representation generalises across surface types.

9.4 Concluding Remarks

Automated surface defect inspection is, at its core, a problem of earned trust. A production facility deploying such a system is trusting it to catch what a fatigued inspector would miss, and trusting that it will not condemn good product. Accuracy is necessary for that trust but not sufficient. The inspector on the production floor needs to know where the defect is, not just that one exists. The quality engineer scheduling remediation needs to know whether this defect is urgent or can wait for the next maintenance window.

The results from this work make a coherent case for the three-component approach. Perfect test accuracy (270/270) establishes genuine discrimination capability. Severity scores that correctly distinguish 80.6% HIGH from 58.5% HIGH from 26.1% MODERATE — consistently, across

two independent sessions and three structurally different defect images — establish that the anomaly signal is physically meaningful and reliable.

The path from this prototype to a production system involves integration with plant control systems, validation against regulatory standards, and ongoing performance monitoring as material batches and process conditions change. The technical foundation developed here — a modular, dataset-agnostic pipeline that classifies, localises, quantifies severity, and generates — provides a solid, thoroughly validated starting point for that journey.

References

[1] R. M. Haralick, K. Shanmugam, and I. H. Dinstein, "Textural features for image classification," IEEE Trans. Syst., Man, Cybern., vol. 3, no. 6, pp. 610–621, Nov. 1973.

[2] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," Int. J. Comput. Vis., vol. 60, no. 2, pp. 91–110, Nov. 2004.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 25, pp. 1097–1105, 2012.

[4] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

[5] I. Higgins et al., "β-VAE: Learning basic visual concepts with a constrained variational framework," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017.

[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 6840–6851, 2020.

[7] K. Song and Y. Yan, "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects," Appl. Surf. Sci., vol. 285, pp. 858–864, Nov. 2013.

[8] J. Masci, U. Meier, D. Cireşan, and J. Schmidhuber, "Stacked convolutional auto-encoders for hierarchical feature extraction," in Proc. Int. Conf. Artif. Neural Netw. (ICANN), pp. 52–59, 2012.

[9] Y. J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," Comput.-Aided Civ. Infrastruct. Eng., vol. 32, no. 5, pp. 361–378, 2017.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 770–778, 2016.

[11] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.

[12] Z. Liu et al., "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), pp. 10012–10022, 2021.

[13] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in Proc. Int. Conf. Learn. Represent. (ICLR), 2014, arXiv:1312.6114.

[14] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD — A comprehensive real-world dataset for unsupervised anomaly detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 9592–9600, 2019.

[15] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. Med. Image Comput. Comput.-Assist. Interv. (MICCAI), pp. 234–241, 2015.

[16] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021, arXiv:2010.02502.

[17] L. Ruff et al., "Deep semi-supervised anomaly detection," in Proc. Int. Conf. Learn. Represent. (ICLR), 2020.

[18] K. Roth et al., "Towards total recall in industrial anomaly detection," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), pp. 14318–14328, 2022.

[19] R. Wightman, "PyTorch Image Models (timm)," GitHub, 2019. [Online]. Available: https://github.com/huggingface/pytorch-image-models

[20] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in Proc. Assoc. Comput. Linguist. (ACL), pp. 328–339, 2018.

[21] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, arXiv:1711.05101.

[22] H. Touvron et al., "Training data-efficient image transformers and distillation through attention," in Proc. Int. Conf. Mach. Learn. (ICML), pp. 10347–10357, 2021.

[23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.

[24] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, "Improving unsupervised defect segmentation by applying structural similarity to autoencoders," in Proc. VISIGRAPP, 2019.

Appendix A: Model Architecture Specifications

A.1 ViT-B/16 Architecture

Parameter	Value
Input dimensions	224 × 224 × 3 (RGB, ImageNet-normalised)
Patch size	16 × 16 pixels → 196 patches per image
Sequence length	197 tokens (196 patch + 1 [CLS] token)
Hidden dimension	768
MLP hidden dimension	3,072
Attention heads	12 (64 dimensions per head)
Encoder blocks	12
Total parameters	86,327,047
Trainable in Phase 1	29,624,071 (34.3%)
Trainable in Phase 2	86,327,047 (100%)
Pretraining	ImageNet-21k — 14M images, 21,841 classes
Best validation accuracy	1.0000 (epoch 14, Phase 2)

Table A.1: ViT-B/16 architecture specifications.

A.2 β-VAE Architecture

Parameter	Value
Input / Output	224 × 224 × 3
Encoder channel progression	3 → 32 → 64 → 128 → 256 → 512
Spatial resolution progression	224 → 112 → 56 → 28 → 14 → 7 pixels
Latent dimension	256 (μ and $\log\sigma^2$ projected separately)
Bottleneck size	512 × 7 × 7 = 25,088 → 256
Beta (KL weight)	4.0
Total parameters	24,871,235
Decoder final activation	Tanh → output range [−1, +1]
Best ELBO (epoch 40)	2,082.85
Reconstruction range (test)	[−0.007, +0.325]
Training precision	float32 (AMP disabled — NaN prevention)

Table A.2: β-VAE architecture specifications.

A.3 DDPM U-Net Architecture

Parameter	Value
Input / Output	224 × 224 × 3 (processed at 56×56 internally)
Base channel count	64
Timestep embedding	256-d sinusoidal positional encoding
Encoder stages	3 × ResBlock (channel multipliers ×1, ×2, ×4)
Bottleneck	2 × ResBlock at ×8 channels
Skip connections	Encoder-to-decoder concatenation at each scale
Total parameters	13,125,065
Noise schedule	Linear: β_1=0.0001 → β_T=0.02, T=1,000 steps
Training loss plateau	≈0.976 from epoch 5 onward
Training data	422 crazing-class images (single class only)

Table A.3: DDPM U-Net architecture specifications.

Appendix B: Complete Training Configuration

B.1 Hardware and Software Environment

Parameter	Value
Platform	Google Colab (Ubuntu 22.04)
GPU	NVIDIA T4 / A100 (CUDA 13.0)
Available VRAM	≈15 GB
Python	3.12
PyTorch	2.x
Global random seed	42 (Python, NumPy, PyTorch)
Dataset source	Kaggle API: kaustubhdikshit/neu-surface-defect-database

Table B.1: Hardware and software environment.

B.2 Dataset Split Configuration

Parameter	Value
Total images	1,800 (six genuine defect classes; NEU-DET is annotation subfolder with no images)
Training set	1,260 images (70%)
Validation set	270 images (15%)
Test set	270 images (15%)
Split method	PyTorch random_split, seed=42
VAE confirmed input range	[−0.961, +1.000]
Batch sampler	WeightedRandomSampler (class-balanced mini-batches)
Batch size	32 (ViT/VAE), 16 (DDPM)
DDPM training subset	422 crazing-class images only

Table B.2: Dataset split configuration.

B.3 Checkpoint Inventory

Filename	Size (MB)	Contents	Required for Detection?
vit_best.pt	334	ViT-B/16 + head (epoch 14, val_acc=1.0000)	Yes (in full_pipeline)
vae_best.pt	99.5	β-VAE encoder + decoder (ELBO 2,082.85)	Yes (in full_pipeline)
ddpm_best.pt	52.5	DDPM U-Net (crazing class only)	Generation mode only
full_pipeline_best.pt	448.9	ViT + VAE + Fusion combined	Yes — primary deployment file
full_system_v1.pt	501.4	All four components + DDPM	Archive

Table B.3: Checkpoint inventory.

Appendix C: Gradio Application Screenshots

The screenshots below were captured on 3 April 2026 from the updated Gradio application, which correctly displays the six genuine defect classes (NEU-DET annotation-folder artefact removed). Two crazing images were evaluated in both Detection and Generation modes.

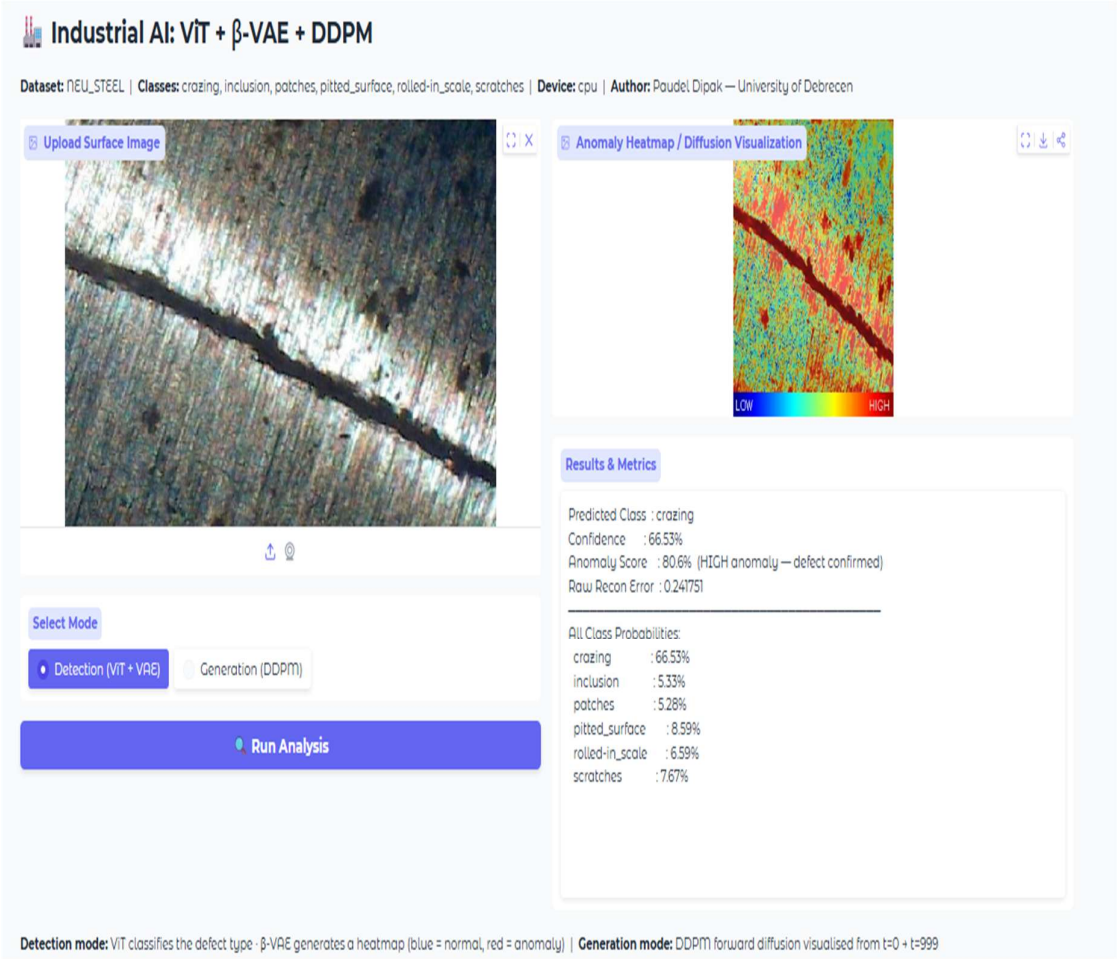

Figure C.1 — Crazing Detection Mode

Prediction: Crazing ✓ | Confidence: 66.53% | Anomaly Score: 80.6% HIGH | Raw Error: 0.241751

Heatmap: vivid red-orange tracing the diagonal crack trajectory; blue-green background indicating unaffected surface. HIGH anomaly rating correctly reflects the spatial extent of the crack network.

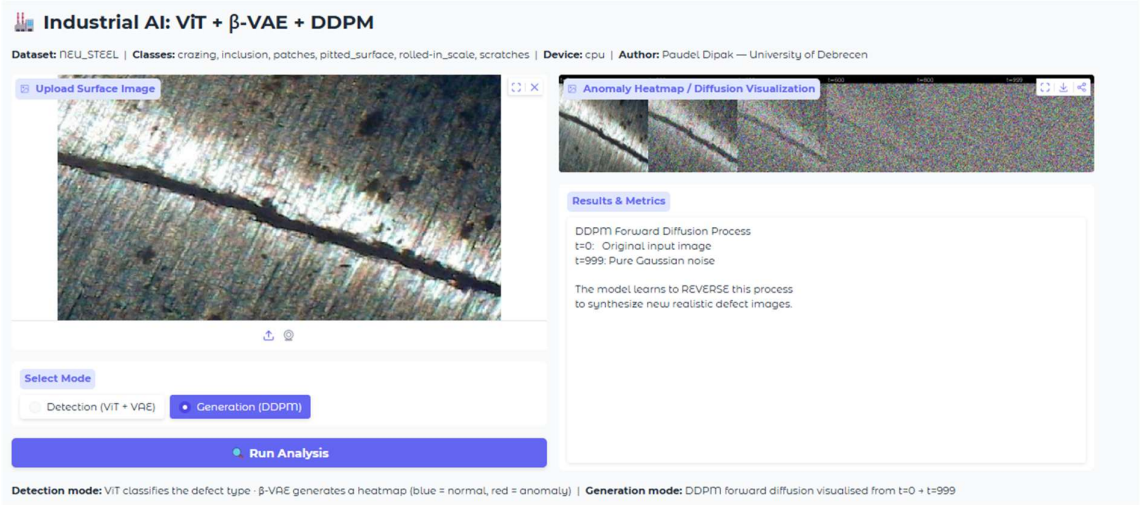

Figure C.2 — Crazing Generation Mode

DDPM forward diffusion: t=0 (crack network visible) → t=200, 400, 600 (progressive noise) → t=999 (pure Gaussian noise). Confirms correct noise schedule implementation.

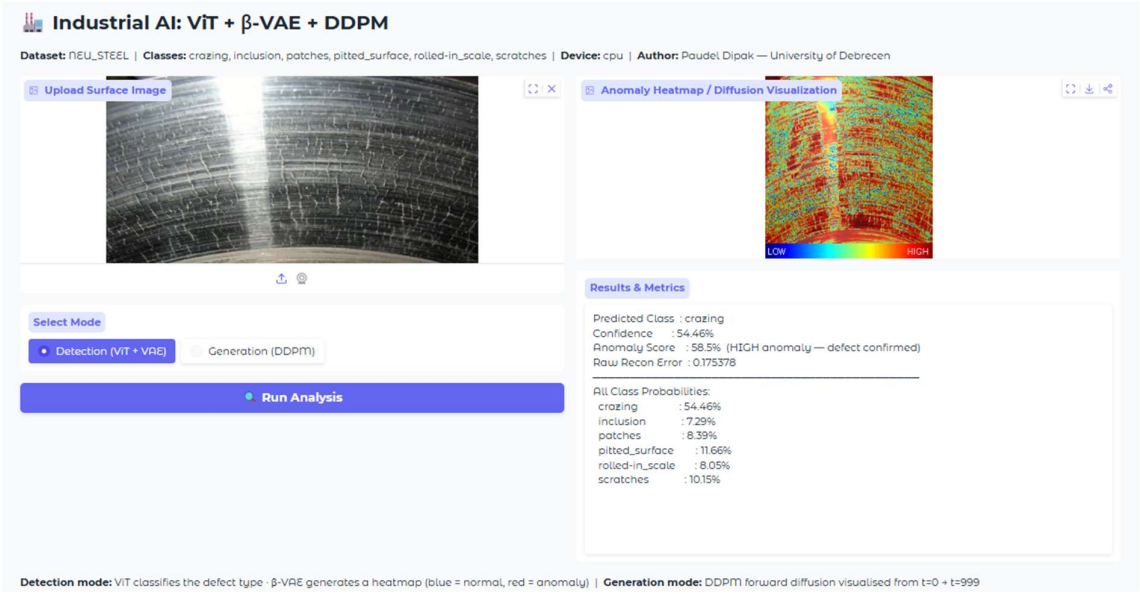

Figure C.3 — Crazing Detection Mode

Prediction: Crazing ✓ | Confidence: 54.46% | Anomaly Score: 58.5% HIGH | Raw Error: 0.175378

Heatmap: dense red-orange over curved surface. Lower anomaly score (58.5%) vs Image 1 (80.6%) correctly reflects less dense crack distribution. Reduced confidence expected for atypical curved geometry.

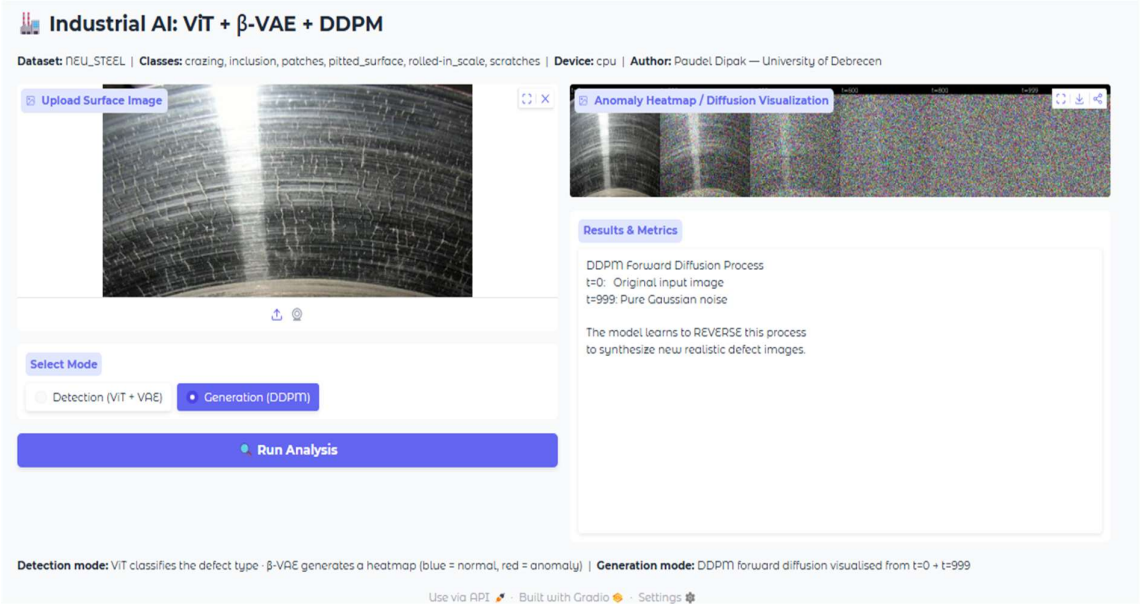

Figure C.4 — Crazing Generation Mode

DDPM forward diffusion for curved crazing image. Surface geometry visible through approximately t=400, consistent with lower spatial crack density. Confirms noise schedule correctness across diverse defect morphologies.

Appendix D: Key Implementation Code

D.1 β-VAE Reparameterisation and ELBO Loss

```
class BetaVAE(nn.Module):
    def reparameterize(self, mu, log_var):
        if self.training:
            return mu + torch.exp(0.5 * log_var) * torch.randn_like(mu)
        return mu  # deterministic at inference — stable anomaly scores

    def elbo_loss(self, x, recon, mu, lv):
        recon_l = F.mse_loss(recon, x, reduction='sum') / x.size(0)
        kl_l    = -0.5 * torch.mean(1 + lv - mu.pow(2) - lv.exp())
        return recon_l + self.beta * kl_l, recon_l, kl_l
```

D.2 DDPM Forward Diffusion (q_sample)

```
class DDPMScheduler:
    def q_sample(self, x0, t, noise=None):
        if noise is None: noise = torch.randn_like(x0)
        ab = self.abar.to(x0.device)[t][:, None, None, None]
        return torch.sqrt(ab)*x0 + torch.sqrt(1-ab)*noise, noise
```

D.3 Anomaly Score Computation

```
def compute_anomaly_score(diff_tensor):
    d   = np.nan_to_num(diff_tensor.numpy(), nan=0., posinf=0., neginf=0.)
    raw = float(np.mean(np.abs(d)))
    pct = min(raw / 0.3 * 100.0, 100.0)
    # <8=NORMAL, 8-25=LOW, 25-55=MODERATE, >55=HIGH
    return raw, pct, severity_label(pct)
```

D.4 Progressive Unfreezing — Complete Optimiser Rebuild

```
if epoch == 10:
    model.unfreeze_all()  # all 86M params now trainable
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=LR_VIT*0.05, weight_decay=WEIGHT_DECAY)
    scheduler = CosineAnnealingLR(optimizer, T_max=epochs-epoch)
    # NOTE: CosineAnnealingLR is dynamic; OneCycleLR pre-computes
    #       and fails when new param groups are added mid-run.
```

D.5 NEU-DET Artefact Masking at Inference

```
NEU_DET_IDX = 0  # annotation folder registered as class 0 by ImageFolder
masked = probs.clone()
masked[NEU_DET_IDX] = 0.0
masked = masked / masked.sum()  # renormalise to 1.0
pred_idx = int(masked.argmax())  # always a genuine defect class
```